GUÍA DE REPOSITORIO GITHUB TEL223

Paso 1 — Crear el repositorio (una sola vez)

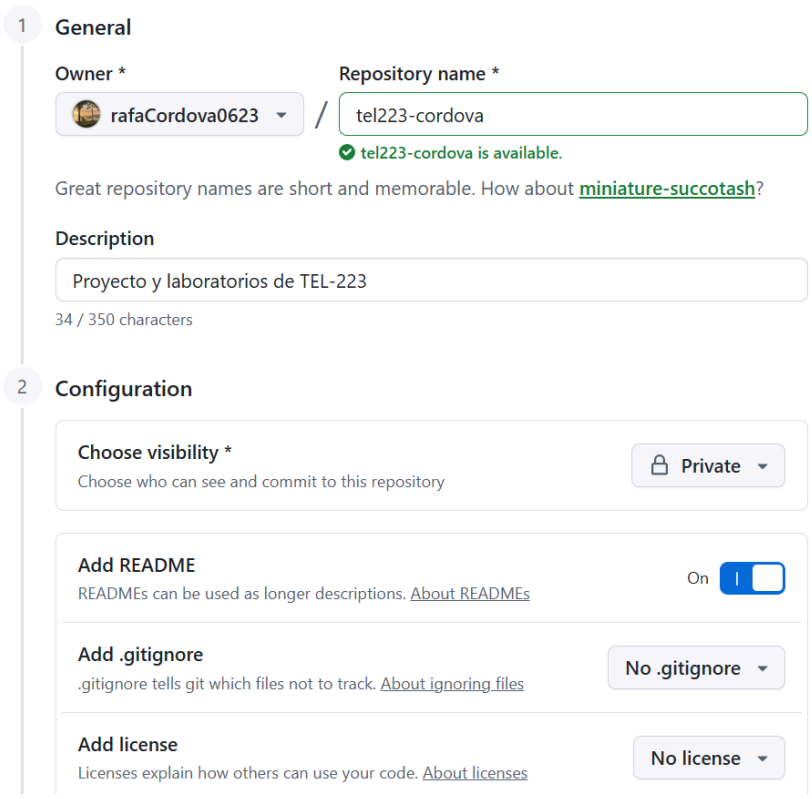
Esto se hace una vez en todo el semestre. Si ya lo tiene creado, salte al paso 2.

1. Entre a su cuenta de GitHub en el navegador y presione el botón New para crear un repositorio nuevo.

2. Complete así:

Campo	Qué poner
Repository name	tel223-apellido (por ejemplo, tel223-mamani)
Description	Proyecto y laboratorios de TEL-223
Visibilidad	PRIVATE. Es obligatorio que sea privado
Add a README file	Márquelo: así el repositorio no queda vacío

3. Presione Create repository. Ya tiene su repositorio en GitHub.



Agregar al docente como colaborador

Como el repositorio es privado, el docente no puede verlo hasta que usted lo agregue. Hágalo apenas lo cree.

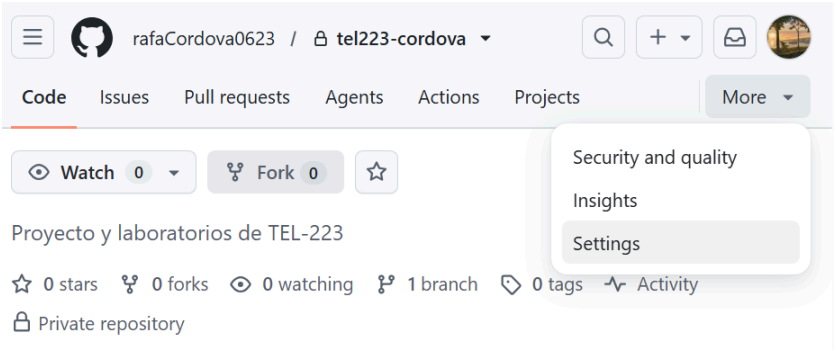
4. Dentro de su repositorio, vaya a Settings, luego a Collaborators.

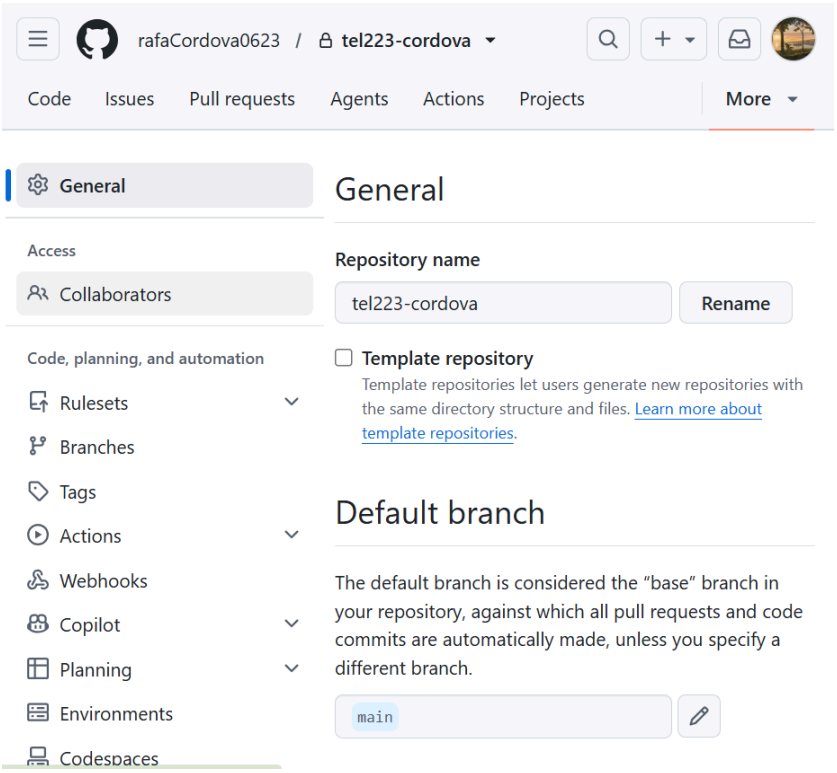
5. Presione Add people y escriba el usuario del docente:

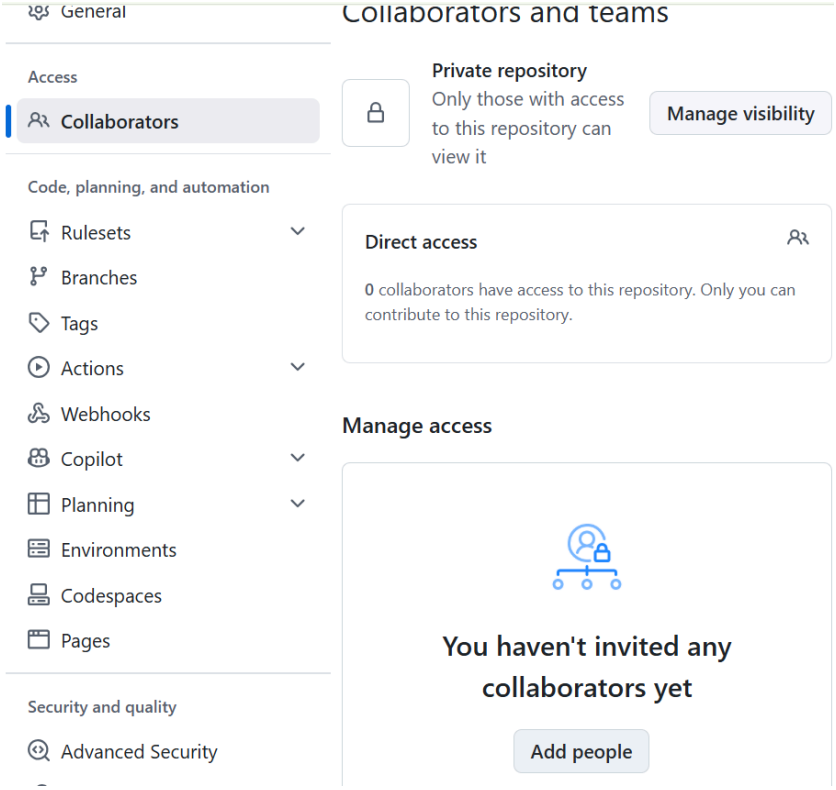
`[tu-usuario-docente]`

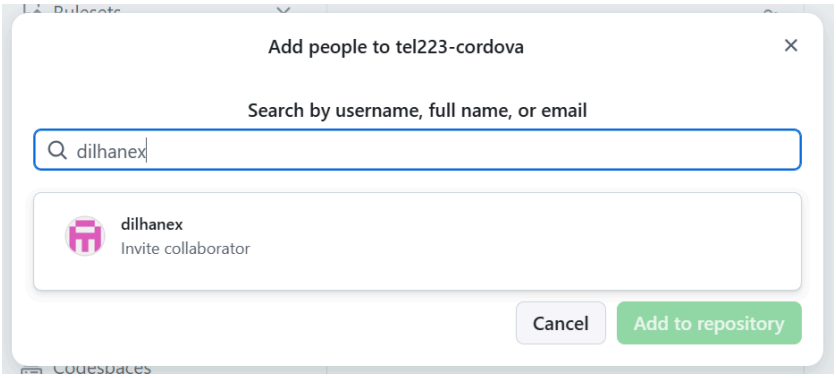
Importante: sin este paso, el docente no puede revisar su trabajo. Es lo primero que se verifica.

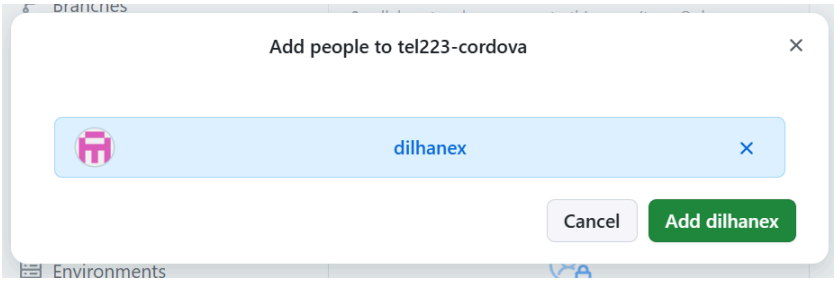
Fin del paso 1 — su repositorio está creado, es privado, y el docente está agregado como colaborador.

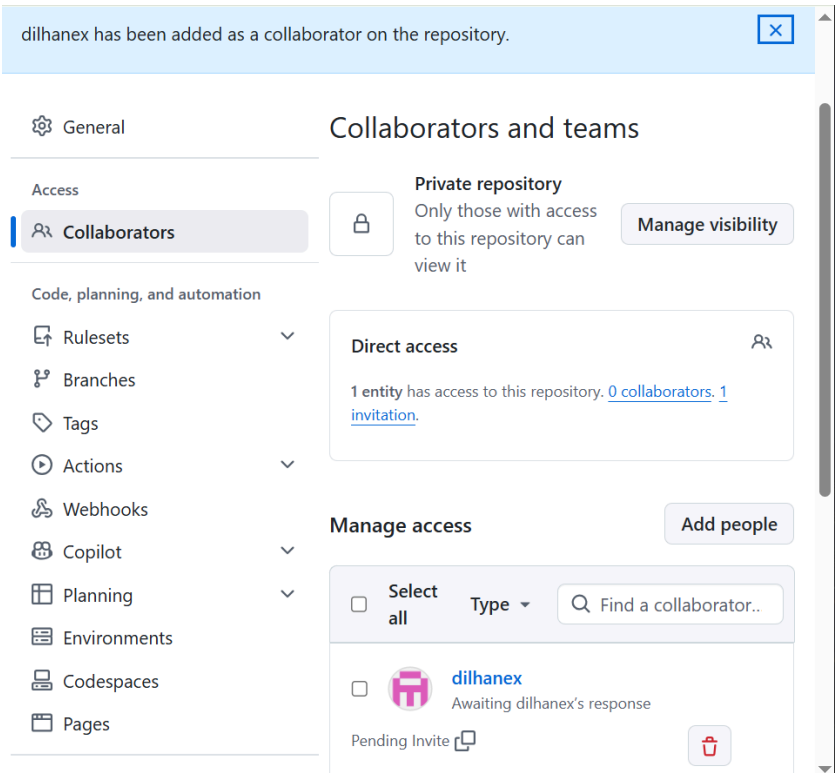

rafaCordova0623 / tel223-cordova
Code
Issues
Pull requests
Agents
Actions
Projects
More
Security and quality
Insights
Settings
Watch 0
Fork 0
Proyecto y laboratorios de TEL-223
0 stars
0 forks
0 watching
1 branch
0 tags
Activity
Private repository


rafaCordova0623 / tel223-cordova
Code
Issues
Pull requests
Agents
Actions
Projects
More
General
Access
Collaborators
Code, planning, and automation
Rulesets
Branches
Tags
Actions
Webhooks
Copilot
Planning
Environments
Codespaces
General
Repository name
tel223-cordova
Rename
Template repository
Template repositories let users generate new repositories with the same directory structure and files. Learn more about template repositories.
Default branch
The default branch is considered the “base” branch in your repository, against which all pull requests and code commits are automatically made, unless you specify a different branch.
main


General
Access
Collaborators
Code, planning, and automation
Rulesets
Branches
Tags
Actions
Webhooks
Copilot
Planning
Environments
Codespaces
Pages
Security and quality
Advanced Security
Collaborators and teams
Private repository
Only those with access to this repository can view it
Manage visibility
Direct access
0 collaborators have access to this repository. Only you can contribute to this repository.
Manage access
You haven't invited any collaborators yet
Add people


Add people to tel223-cordova
Search by username, full name, or email
dilhanex
dilhanex
Invite collaborator
Cancel
Add to repository


Add people to tel223-cordova
dilhanex
Cancel
Add dilhanex



PASO 2 - Traer el repositorio a su computadora

Paso 2 — Traer el repositorio a su computadora

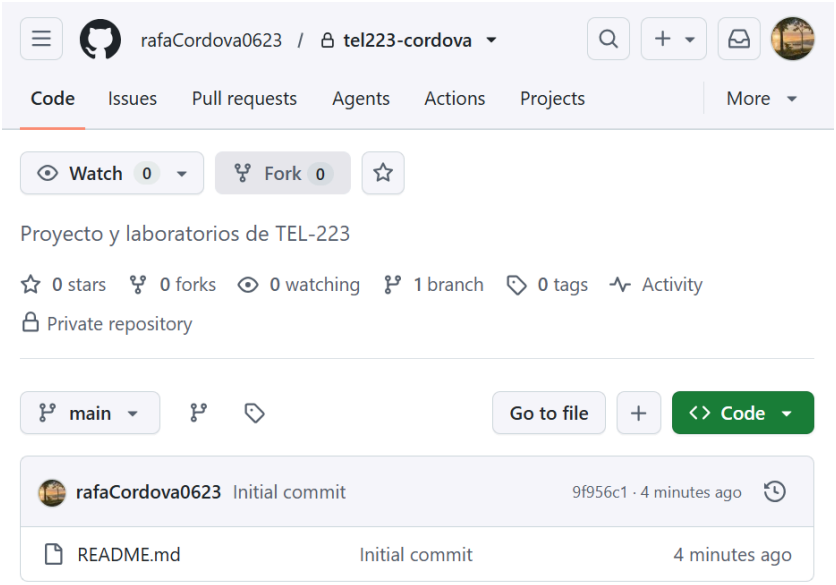
Para trabajar cómodo, traiga una copia del repositorio a su equipo. Esto se hace una vez por computadora.

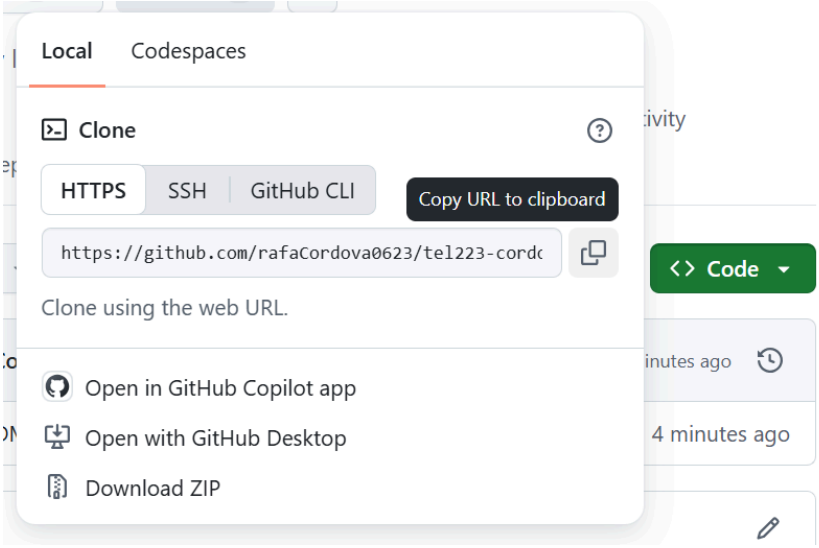
6. En la página de su repositorio, presione el botón verde Code y copie la dirección que aparece (la que empieza con https).

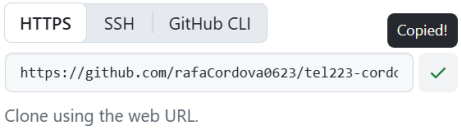
7. Abra Git Bash y clone el repositorio, pegando esa dirección:

```
git clone https://github.com/su-usuario/tel223-apellido.git
```

Qué acaba de hacer: descargó una carpeta con su repositorio. A partir de ahora trabaja dentro de esa carpeta.


rafaCordova0623 / tel223-cordova
Code
Issues
Pull requests
Agents
Actions
Projects
More
Watch 0
Fork 0
Proyecto y laboratorios de TEL-223
0 stars
0 forks
0 watching
1 branch
0 tags
Activity
Private repository
main
Go to file
Code
rafaCordova0623 Initial commit
9f956c1 · 4 minutes ago
README.md
Initial commit
4 minutes ago


Local
Codespaces
Clone
HTTPS
SSH
GitHub CLI
Copy URL to clipboard
https://github.com/rafaCordova0623/tel223-cordc
Clone using the web URL.
Open in GitHub Copilot app
Open with GitHub Desktop
Download ZIP
Code
4 minutes ago


HTTPS
SSH
GitHub CLI
Copied!
https://github.com/rafaCordova0623/tel223-cordc
Clone using the web URL.



Explicación del error:

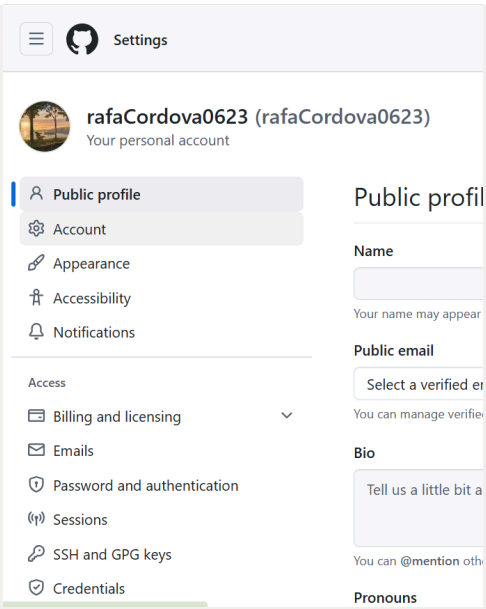
💡 Perfecto, vamos a explicarlo desde cero, sin asumir que ya conoces GitHub ni Git.
El error que viste no es porque hayas hecho algo mal en Git Bash, sino porque **GitHub ya no acepta usuario y contraseña normales para clonar repositorios**. Ahora exige otro método de autenticación.

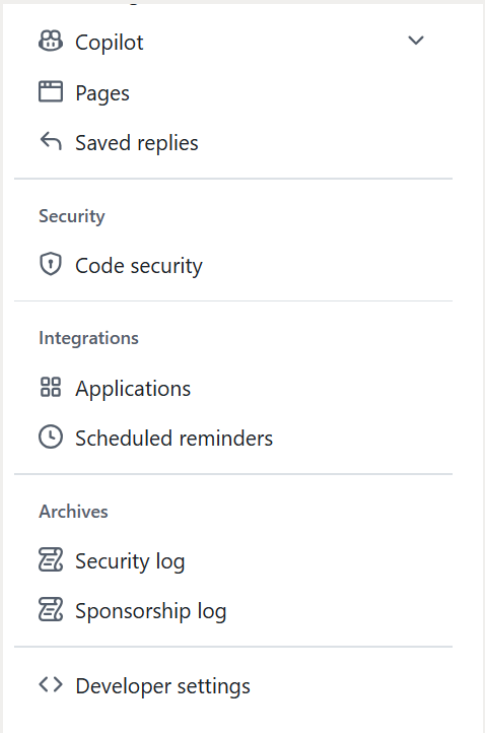
🚀 Paso a paso para solucionar

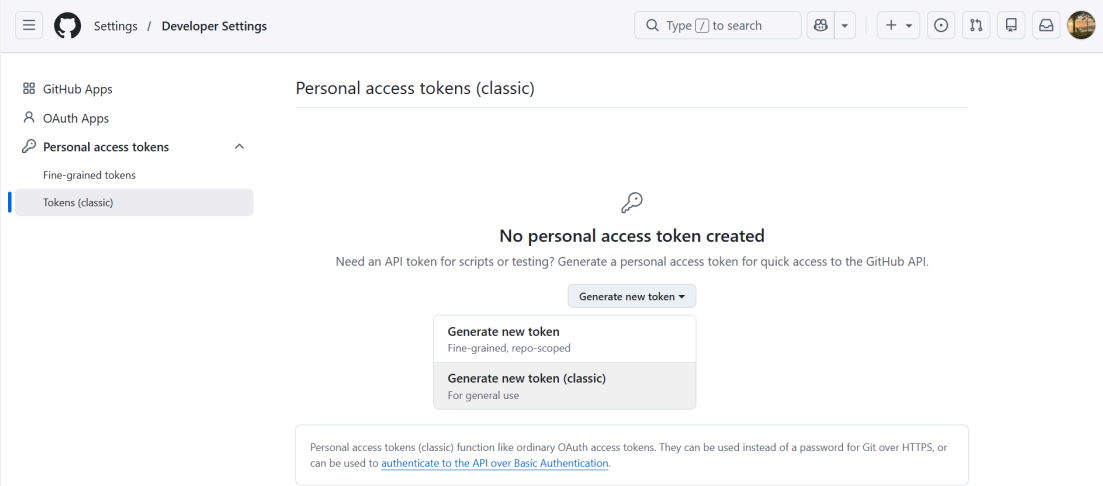
1. Clonar con HTTPS usando un token personal

Este método es el más sencillo si recién empiezas.

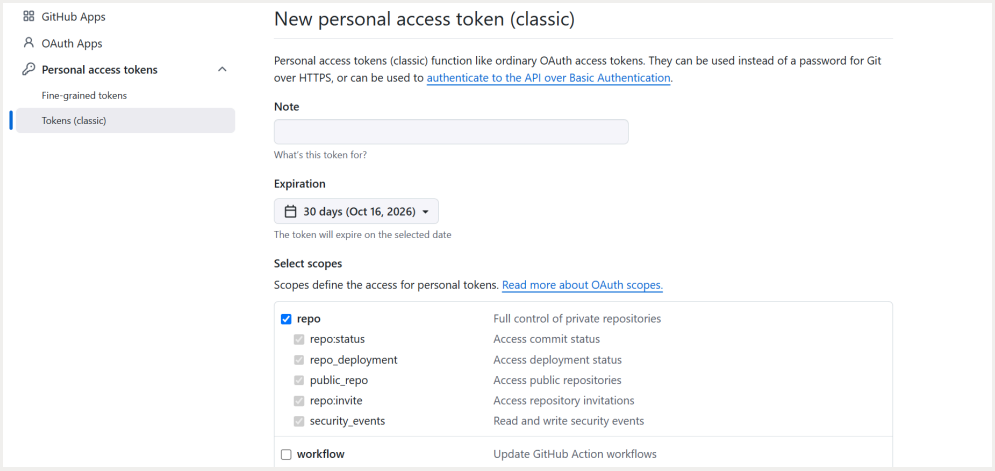
1. Entra a tu cuenta de GitHub.
2. Ve a **Settings → Developer settings (última opción del menú de la izquierda)→ Personal access tokens → Tokens (classic)**.


Settings
rafaCordova0623 (rafaCordova0623)
Your personal account
Public profile
Account
Appearance
Accessibility
Notifications
Access
Billing and licensing
Emails
Password and authentication
Sessions
SSH and GPG keys
Credentials
Public profil
Name
Your name may appear
Public email
Select a verified e
You can manage verifie
Bio
Tell us a little bit a
You can @mention oth
Pronouns


Copilot
Pages
Saved replies
Security
Code security
Integrations
Applications
Scheduled reminders
Archives
Security log
Sponsorship log
Developer settings



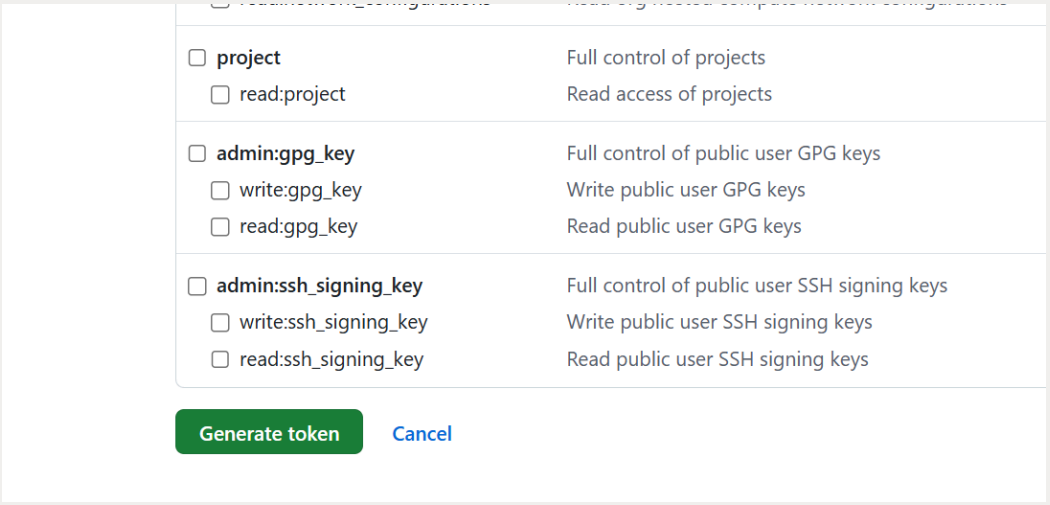
1. Crea un nuevo token:
    - Dale permisos de `repo`.



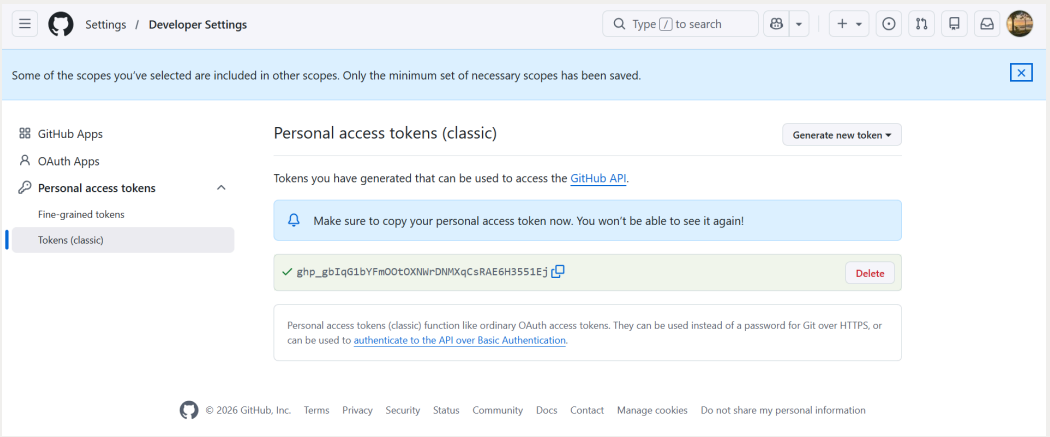
    En el campo **“Note”** puedes escribir cualquier texto breve que te ayude a identificar ese token más adelante. No es un dato técnico ni afecta el funcionamiento del token, simplemente es una etiqueta descriptiva.
    Por ejemplo, podrías poner:

    - `Token TEL223`
    - `Clases Git septiembre 2026`
    - `Repositorio cordova`
    - `Uso personal laptop`

👉 Lo importante es que no quede vacío, porque GitHub te obliga a darle un nombre. Ese nombre es solo para ti, para que sepas por qué generaste ese token.



- Copia ese token (es como una contraseña especial).



→ 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 (token)

1. En Git Bash, cuando hagas:
   bash

```
git clone https://github.com/tu-usuario/tel223-apellido.git
```

Pero como ya realicé ese comando antes, la carpeta ya existe en el pc pese a no haberse clonado con éxito. Es por tal motivo, que se eliminó la carpeta y se volvió a clonar el repositorio con el token generado previamente:

🗑️ 1. Borrar la carpeta existente

En Git Bash, escribe este comando (asegúrate de estar en la carpeta donde está `tel223-practica-git-cordova`):
bash

```
rm-rftel223-practica-git-cordova
```

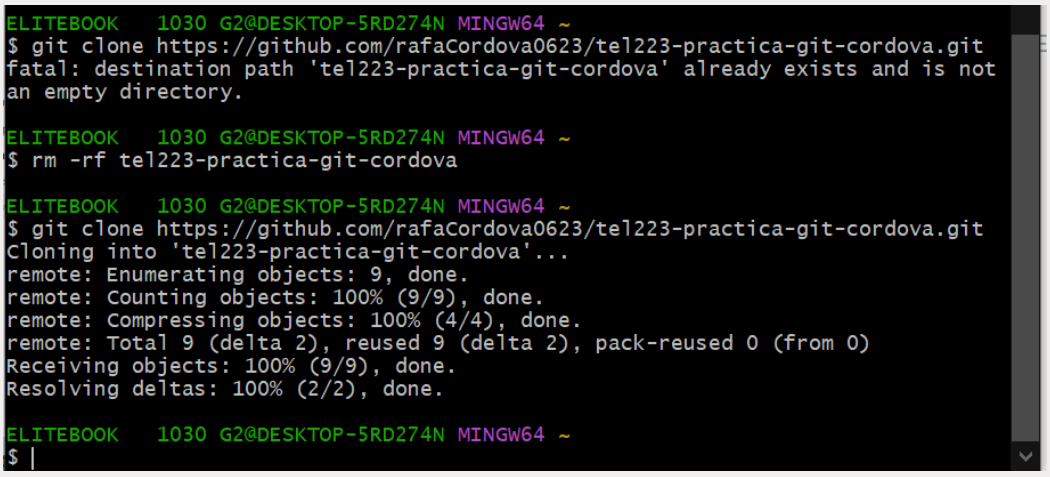
- `rm` = remove (eliminar)
- `rf` = borra todo el contenido sin pedir confirmación
  ⚠️ Esto eliminará la carpeta y todo lo que tenga dentro.

📥 2. Volver a clonar el repositorio

Ahora sí, ejecuta el comando de clonación:
bash

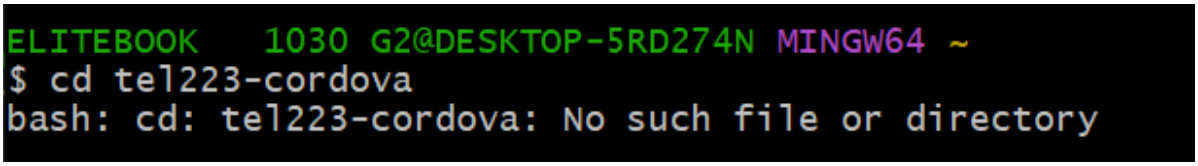
```
gitclonehttps://github.com/rafaCordova0623/tel223-practica-git-cordova.git
```

Esto creará nuevamente la carpeta `tel223-practica-git-cordova` con el contenido del repositorio.



8. Entre a la carpeta:

```
cd tel223-apellido
```



💡 **Cómo resolverlo**

1. **Verifica qué carpetas tienes en tu directorio actual**
   En Git Bash, escribe:
   bash

   ```
   ls
   ```

   Esto mostrará las carpetas y archivos que tienes. Busca si aparece `tel223-cordova` o algún nombre parecido.

```
ELITEBOOK   1030 G2@DESKTOP-5RD274N MINGW64 ~
$ ls
'3D Objects'/
 AndroidStudioProjects/
 AppData/
'Cisco Packet Tracer 8.2.2'/
'Configuración local'@
 Contacts/
 Cookies@
'Creative Cloud Files'/
'Creative Cloud Files  rafaela.cordova.almendras@gmail.com 23
472A0F641CBEC70A495EF0@AdobeID'/
```

```
'Datos de programa'@
 Desktop/
 Documents/
 Downloads/
'Entorno de red'@
 Favorites/
 Impresoras@
 IntelGraphicsProfiles/
 Links/
'Menú Inicio'@
 MicrosoftEdgeBackups/
'Mis documentos'@
'Muse Hub'/
 Music/
 NTUSER.DAT
 NTUSER.DAT{e7cb3097-2e70-11eb-b2ac-c4d9872b1326}.TxR.0.regtr
ans-ms
 NTUSER.DAT{e7cb3097-2e70-11eb-b2ac-c4d9872b1326}.TxR.1.regtr
ans-ms
 NTUSER.DAT{e7cb3097-2e70-11eb-b2ac-c4d9872b1326}.TxR.2.regtr
ans-ms
 NTUSER.DAT{e7cb3097-2e70-11eb-b2ac-c4d9872b1326}.TxR.blf
 NTUSER.DAT{e7cb3098-2e70-11eb-b2ac-c4d9872b1326}.TM.blf
 NTUSER.DAT{e7cb3098-2e70-11eb-b2ac-c4d9872b1326}.TMContainer
00000000000000000001.regtrans-ms
 NTUSER.DAT{e7cb3098-2e70-11eb-b2ac-c4d9872b1326}.TMContainer
00000000000000000002.regtrans-ms
 OneDrive/
 Pictures/
 Plantillas@
 Praat/
 Reciente@
 STM32Cube/
 STM32CubeIDE/
'Saved Games'/
 Searches/
```

```
 SendTo@
 Sti_Trace.log
 Videos/
'VirtualBox VMs'/
'WPS Cloud Files'/
 blenderkit_data/
 developer-16-abril/
 laboratorio1/
 mentalmentor/
 mi-app/
 ntuser.dat.LOG1
 ntuser.dat.LOG2
 ntuser.ini
 source/
 tel223-practica-git-cordova/
 test-dir/
```

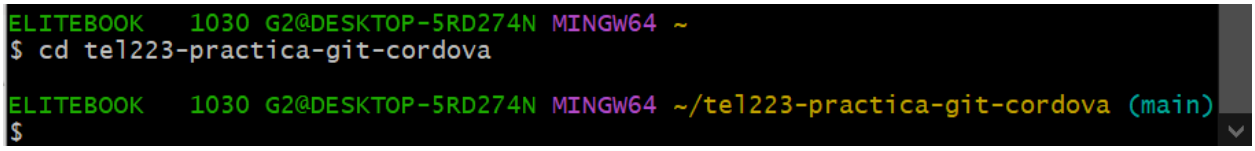
💡 Muy bien, ya encontraste la carpeta correcta: `tel223-practica-git-cordova`.
El error anterior se debía a que intentabas entrar a `tel223-cordova`, pero esa carpeta no existe.

Paso que debes seguir ahora

Entrar a la carpeta del repositorio clonado

bash

```
cd tel223-practica-git-cordova
```



9. Configure su identidad, solo la primera vez, para que sus commits lleven su nombre:

```
git config user.name "Su Nombre"
git config user.email "su-correo@ucb.edu.bo"
```

```
ELITEBOOK   1030 G2@DESKTOP-5RD274N MINGW64 ~/tel223-practica-git-cordova (main)
$ git config user.name "rafaCordova0623"

ELITEBOOK   1030 G2@DESKTOP-5RD274N MINGW64 ~/tel223-practica-git-cordova (main)
$ git config user.email "rafaela.cordova@ucb.edu.bo"
```

PASO 3 - Organizar las carpetas

Paso 3 — Organizar las carpetas

Una estructura simple para que todo esté ordenado y el docente encuentre las cosas rápido.

Dentro de la carpeta de su repositorio, cree estas dos carpetas:

```
mkdir proyecto
mkdir laboratorios
```

Carpeta	Qué va adentro
proyecto	Los avances de su proyecto integrador: ficha, diagrama, documentos, código cuando empiece
laboratorios	Las evidencias de cada laboratorio: capturas, bitácoras, archivos de configuración

Consejo: dentro de laboratorios, cree una subcarpeta por cada uno: lab3, lab4, y así. Así no se mezclan las evidencias.

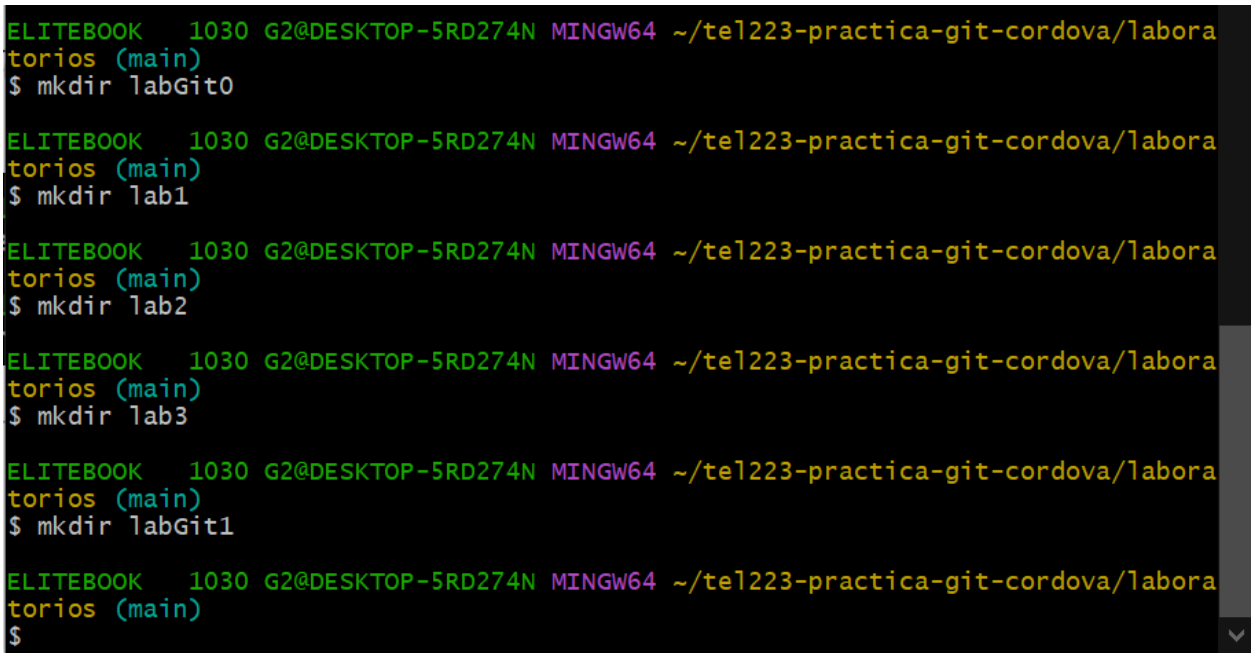
```
ELITEBOOK   1030 G2@DESKTOP-5RD274N MINGW64 ~/tel223-practica-git-cordova (main)
$ mkdir proyecto

ELITEBOOK   1030 G2@DESKTOP-5RD274N MINGW64 ~/tel223-practica-git-cordova (main)
$ mkdir laboratorios
```

Creación de subcarpetas:

Donde:

- labGit0 → Representa a la primera práctica introductorio a Git con propósito de crear un repositorio local y en GitHub.
- labGit1 → Representa la guía actual de creación del repositorio *tel223-practica-git-cordova*.


ELITEBOOK 1030 G2@DESKTOP-5RD274N MINGW64 ~/tel223-practica-git-cordova/laboratorios (main)
$ mkdir labGit0
ELITEBOOK 1030 G2@DESKTOP-5RD274N MINGW64 ~/tel223-practica-git-cordova/laboratorios (main)
$ mkdir lab1
ELITEBOOK 1030 G2@DESKTOP-5RD274N MINGW64 ~/tel223-practica-git-cordova/laboratorios (main)
$ mkdir lab2
ELITEBOOK 1030 G2@DESKTOP-5RD274N MINGW64 ~/tel223-practica-git-cordova/laboratorios (main)
$ mkdir lab3
ELITEBOOK 1030 G2@DESKTOP-5RD274N MINGW64 ~/tel223-practica-git-cordova/laboratorios (main)
$ mkdir labGit1
ELITEBOOK 1030 G2@DESKTOP-5RD274N MINGW64 ~/tel223-practica-git-cordova/laboratorios (main)
$

PASO 4

Paso 4 — Subir su trabajo (esto se repite siempre)

Este es el ciclo que va a repetir cada vez que tenga algo que entregar. Son cuatro comandos y siempre los mismos.

10. Copie sus archivos a la carpeta que corresponda (proyecto o laboratorios) usando el explorador de Windows, como cualquier archivo.

11. En Git Bash, dentro de la carpeta del repositorio, agregue todo lo nuevo:

```
git add .
```

Qué hace: marca todos los archivos nuevos o modificados para guardarlos. El punto significa todo.

12. Guarde el cambio con un mensaje que diga qué subió:

```
git commit -m "Evidencia del laboratorio 3"
```

El mensaje importa: el docente lo lee para saber qué entregó. Sea claro: “Avance del proyecto: diagrama”, “Evidencia lab 3”.

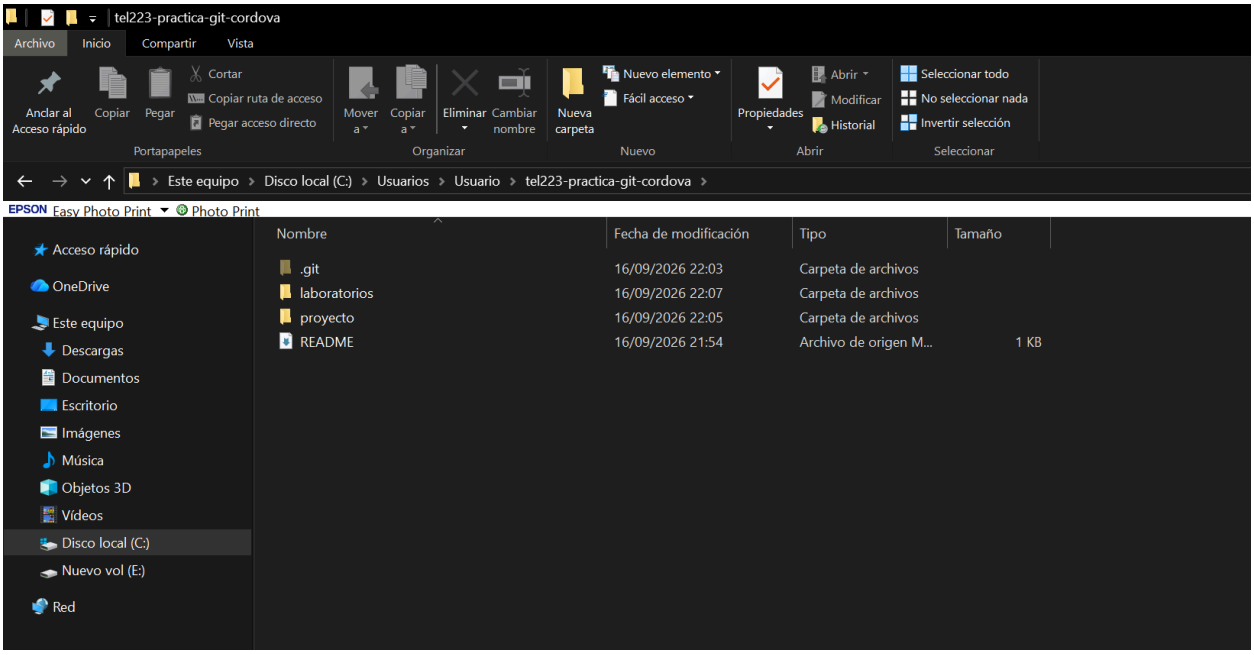
13. Envíe sus cambios a GitHub:

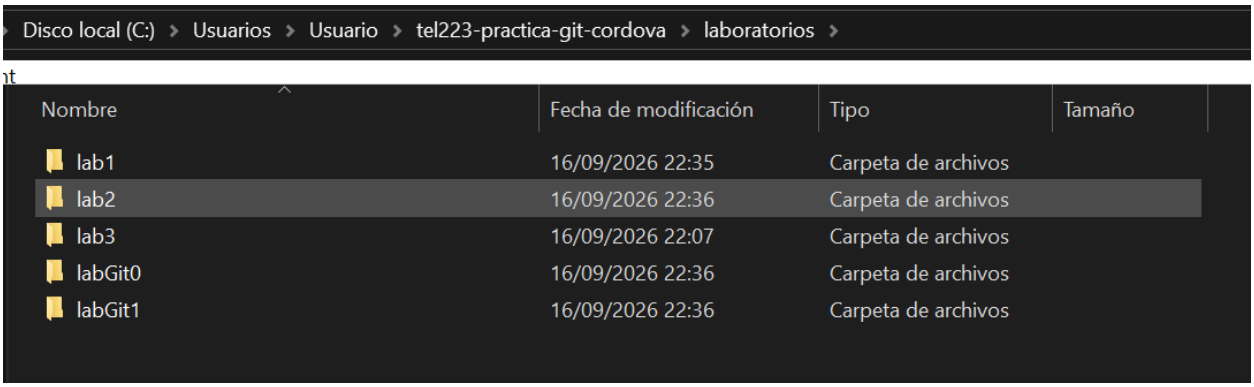
```
git push
```

Qué debe ver: un resumen de la transferencia. Recargue la página de su repositorio en el navegador y verá sus archivos publicados.

Si le pide usuario y contraseña: GitHub ya no acepta la contraseña normal. Use un token de acceso personal en lugar de la contraseña. Vea la sección de ayuda al final.

El ciclo completo — cambio + git add + git commit + git push. Siempre esos cuatro pasos, en ese orden. Es todo lo que necesita para entregar.






```
ELITEBOOK   1030 G2@DESKTOP-5RD274N MINGW64 ~/tel223-practica-git-cordova/labora
torios (main)
$ cd ..

ELITEBOOK   1030 G2@DESKTOP-5RD274N MINGW64 ~/tel223-practica-git-cordova (main)
$ git add .

ELITEBOOK   1030 G2@DESKTOP-5RD274N MINGW64 ~/tel223-practica-git-cordova (main)
$
```

```
ELITEBOOK   1030 G2@DESKTOP-5RD274N MINGW64 ~/tel223-practica-git-cordova (main)
$ git commit -m "Evidencia del laboratorio 3"
[main 14d4506] Evidencia del laboratorio 3
 6 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 laboratorios/lab1/Guia Laboratorio1 - TEL223 - Linux.pdf
 create mode 100644 laboratorios/lab1/lab1evidencia.pdf.pdf
 create mode 100644 "laboratorios/lab2/Guia Practica 2 -  SERVICIOS, REGISTROS Y
 DIAGN\303\223STICO - TEL223 (LLENADO DE DOCUMENTO).pdf"
 create mode 100644 laboratorios/lab2/lab2evidencia.pdf.pdf
 create mode 100644 laboratorios/labGit0/Practica 1 TEL223 - CORDOVA ALMENDRAS,
RAFAELA (Entregable Final).pdf
 create mode 100644 laboratorios/labGit1/Guia Repositorio GitHub TEL223.pdf

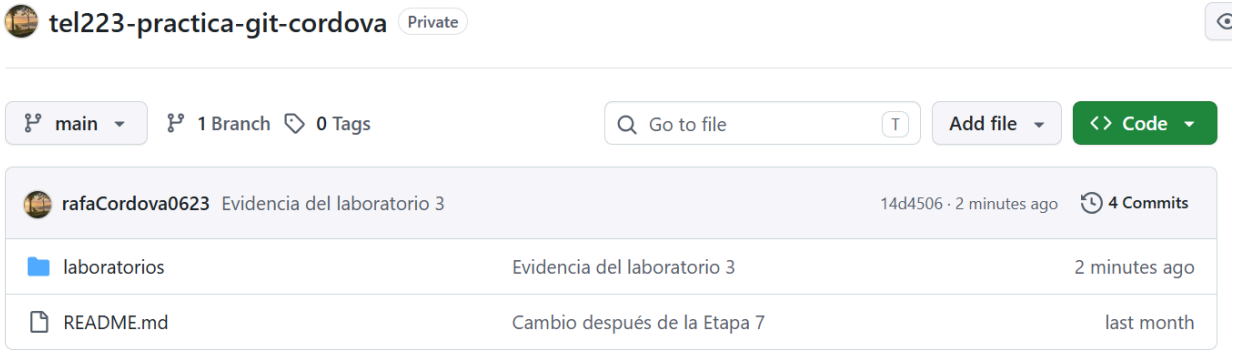
ELITEBOOK   1030 G2@DESKTOP-5RD274N MINGW64 ~/tel223-practica-git-cordova (main)
$
```

```
ELITEBOOK   1030 G2@DESKTOP-5RD274N MINGW64 ~/tel223-practica-git-cordova (main)
$ git push
Enumerating objects: 14, done.
Counting objects: 100% (14/14), done.
Delta compression using up to 4 threads
Compressing objects: 100% (13/13), done.
Writing objects: 100% (13/13), 4.80 MiB | 915.00 KiB/s, done.
Total 13 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/rafaCordova0623/tel223-practica-git-cordova.git
   97e98f1..14d4506  main -> main

ELITEBOOK   1030 G2@DESKTOP-5RD274N MINGW64 ~/tel223-practica-git-cordova (main)
$
```

Verificación: ¿está todo bien?

Mi repositorio existe en GitHub y es privado.	☑
Agregué al docente como colaborador.	☑
Cloné el repositorio en mi computadora.	☑
Tengo las carpetas proyecto y laboratorios.	☑
Sé hacer el ciclo git add, git commit, git push.	☑
Subí al menos una evidencia y la veo publicada en GitHub.	☑
Mis mensajes de commit describen qué subí.	☑



PASO EXTRA → Añadir esta evidencia y la guía de Git nueva llenada

💡 En tu caso, lo más sencillo y habitual es usar:

```
git add .

git commit -m "Añado dos archivos nuevos"

git push
```

desde la carpeta principal del repositorio:

```
~/tel223-practica-git-cordova
```

¿Qué ocurre con `git add .`?

Si ejecutas:

```
git add .
```

en `tel223-practica-git-cordova`, Git buscará cambios en todas las subcarpetas del repositorio y añadirá los archivos nuevos, modificados o eliminados.
Por ejemplo, si has añadido:

```
tel223-practica-git-cordova

└── laboratorios

└── labGit1

├── archivo1.pdf

└── archivo2.docx
```

entonces:

```
cd ~/tel223-practica-git-cordova

git add .
```

añadirá ambos archivos automáticamente.

¿Es necesario entrar en la subcarpeta?

No.
Puedes permanecer en:

```
~/tel223-practica-git-cordova
```

y ejecutar:

```
git add .

git commit -m "Agrego archivos del laboratorio"

git push
```

Esta es la forma que utilizan la mayoría de estudiantes en prácticas sencillas.

¿Y si quisiera añadir sólo dos archivos concretos? [OPCIÓN REALIZADA]

Entonces sí podrías especificarlos.
Por ejemplo:

```
git add laboratorios/labGit1/archivo1.pdf

git add laboratorios/labGit1/archivo2.docx
```

o ambos a la vez: [LO QUE ELEGÍ]

```
git add laboratorios/labGit1/archivo1.pdf laboratorio
s/labGit1/archivo2.docx
```

¿Cómo saber qué archivos se van a subir?

Antes del commit ejecuta:

```
git status
```

Verás algo parecido a:

```
Changes to be committed:

new file: laboratorios/labGit1/archivo1.pdf

new file: laboratorios/labGit1/archivo2.docx
```

Si aparecen los dos archivos que acabas de copiar, entonces puedes continuar con:

```
git commit -m "Agrego archivos del laboratorio"

git push
```

Recomendación para principiantes: sitúate siempre en la carpeta raíz `tel223-practica-git-cordova`, ejecuta primero `git status` para comprobar los cambios y, si sólo has añadido esos dos archivos, usa tranquilamente:

```
git add .

git commit -m "Agrego archivos nuevos"

git push
```

porque es más difícil equivocarse con las rutas.